

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Khoa Công nghệ Thông tin



BÁO CÁO ĐỒ ÁN CUỐI KỲ

Object Detection

Đếm rau củ quả trong băng chuyền

Giảng viên môn học

Thầy Ngô Minh Nhựt

Thầy Lê Long Quốc

Sinh viên thực hiện

Trần Tiến Cường - 23127332

Mục lục

1	Bài toán và bộ dữ liệu	3
1.1	Chọn bài toán	3
1.2	Dữ liệu	3
1.3	Chia tập train / val / test	3
1.4	Thống kê bộ dữ liệu	4
1.4.1	Dữ liệu trông như thế nào	4
1.4.2	Số instance theo lớp	5
1.4.3	Số vật thể trong một ảnh	5
1.4.4	Kích thước vật thể	6
2	Ba kiến trúc được chọn	6
2.1	Lý do lựa chọn mô hình	6
2.2	Cơ chế của từng hướng	6
2.3	Checkpoint mô hình với trọng số COCO	7
2.4	Cấu hình huấn luyện	7
2.5	Thời gian huấn luyện	8
3	Kết quả trên tập test	8
3.1	Bảng tổng hợp	8
3.2	So với checkpoint COCO gốc	8
3.3	AR theo kích thước vật thể	8
3.4	AP theo từng lớp	9
3.5	Quan sát trên tập test	9
4	Đánh giá từng kiến trúc	10
4.1	SSDLite320-MobileNetV3-Large	10
4.2	YOLO11s	10
4.3	D-FINE-N	10
4.4	Huấn luyện lại trên tập con của dữ liệu tiền huấn luyện làm mô hình tệ đi?	11
4.5	Khoảng cách giữa khả năng tìm và khả năng chấm điểm	11
5	Thử nghiệm mở rộng: YOLO26m trên Open Images	12
5.1	Cấu hình	12
5.2	Kết quả	12
5.3	So sánh với ba mô hình chính	12

6	Ứng dụng web	13
6.1	Mô hình được chọn	13
6.2	Kiến trúc hệ thống	13
6.3	Chức năng 1: tải ảnh lên và trả kết quả	14
6.4	Chức năng 2: camera thời gian thực và giả lập đếm bằng chuyên	14
6.5	Lưu và xem lại các phiên	14
6.6	Số đo hiệu năng	16
	Lời cảm ơn	16
	Tài liệu tham khảo	16

1 Bài toán và bộ dữ liệu

1.1 Chọn bài toán

Trong các dây chuyền phân loại và đóng gói nông sản hiện đại, việc đếm, thống kê sản phẩm theo thời gian thực đóng vai trò cốt lõi nhằm tối ưu hóa vận hành. Dù số lượng nông sản trong thực tế rất đa dạng, bài làm này bước đầu xây dựng mô hình phát hiện đối tượng tập trung vào 5 loại rau củ quả phổ biến: táo, chuối, bông cải xanh, cà rốt và cam, tập trung vào đánh giá, nhận xét và khả năng triển khai các kiến trúc mô hình trong bài toán này.

1.2 Dữ liệu

Dữ liệu là tập con của COCO 2017 (gồm 80 lớp, chỉ có khoảng 5 lớp liên quan đến nông sản), lấy qua thư viện `fiftyone`. Lấy và xử lý dữ liệu gồm năm bước, mỗi bước một script:

Script	Việc
<code>scripts/01_download_dataset.py</code>	Duyệt COCO 2017, giữ lại mọi ảnh chứa ít nhất một trong 5 lớp, ghi <code>data/manifest.json</code>
<code>scripts/01b_fetch_images.py</code>	Tải file ảnh về, kiểm tra số byte so với <code>Content-Length</code>
<code>scripts/02_build_dataset.py</code>	Từ manifest sinh ra <code>images/</code> , <code>labels/</code> (định dạng YOLO), <code>annotations/</code> (định dạng COCO) và <code>data.yaml</code>
<code>scripts/03_verify_dataset.py</code>	Đối chiếu nhãn YOLO với nhãn COCO trên cùng một ảnh, phát hiện lệch tọa độ hoặc lệch chỉ số lớp
<code>scripts/04_check_images.py</code>	Ép giải mã toàn bộ pixel của mọi ảnh để bắt file bị cắt ngắn

Một phản hồi HTTP bị đứt giữa chừng vẫn để lại một file JPEG trông hoàn chỉnh, và `Image.verify()` của PIL chỉ đọc phần header nên không phát hiện được. Lỗi chỉ lộ ra khi giải mã toàn bộ pixel, tức là ở epoch huấn luyện đầu tiên. `scripts/04_check_images.py` giải mã trước và tải lại các file hỏng.

Hai định dạng nhãn được sinh song song vì ba thư viện huấn luyện đọc hai chuẩn khác nhau: `ultralytics` đọc file `.txt` kiểu YOLO, còn `torchvision` và `transformers` đọc JSON kiểu COCO.

Kết quả bước dựng dữ liệu, theo `data/manifest.json`:

Nguồn	Ảnh duyệt	Ảnh giữ	Nhãn giữ
COCO train2017	6.827	6.827	36.868
COCO val2017	310	310	1.592

1.3 Chia tập train / val / test

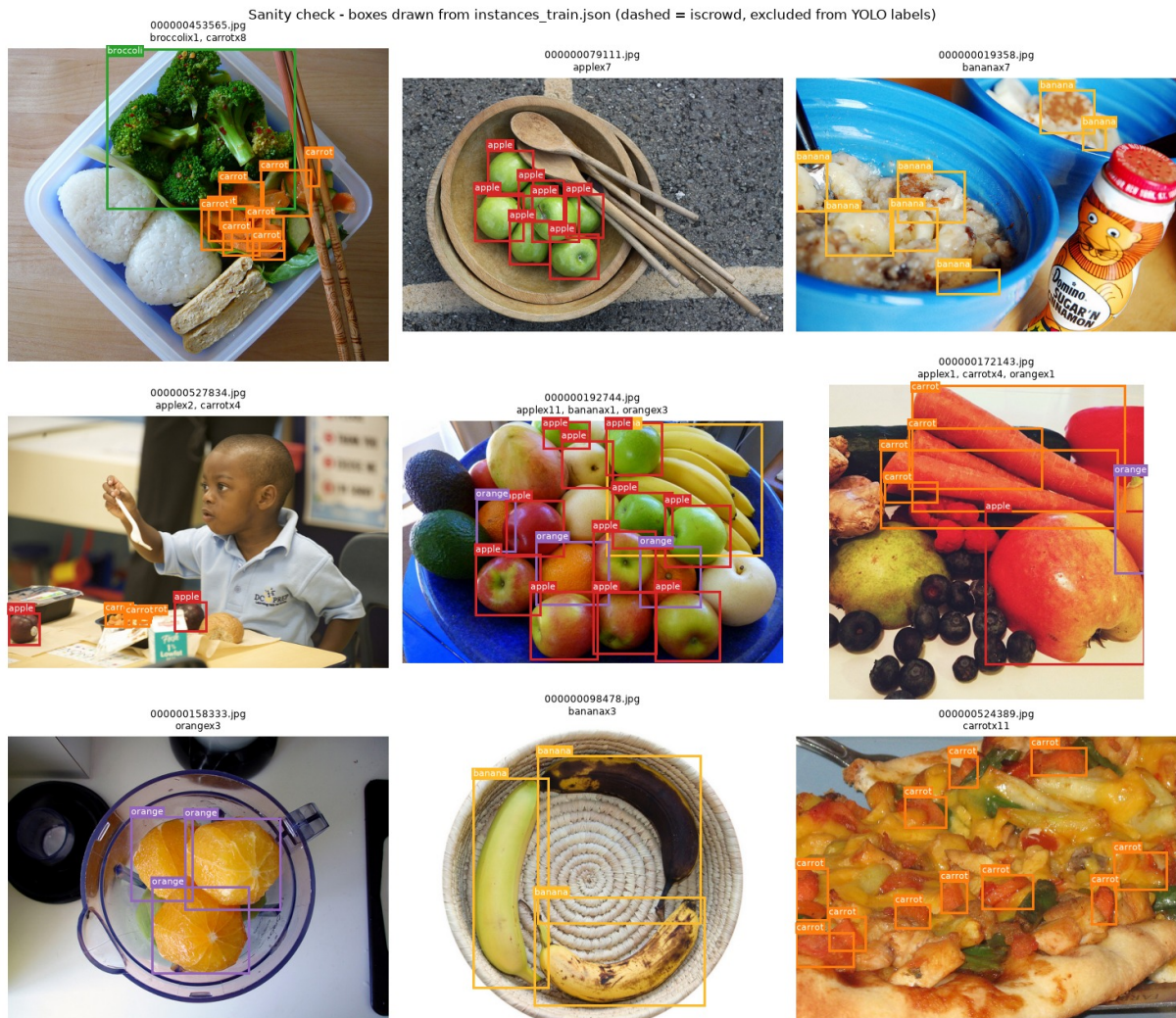
Nguồn: `data/splits.json`, `data/annotations/instances_<split>.json`

Split	Ảnh	Instance (thô)	Instance (dùng để huấn luyện)	Nguồn
train	5.803	31.389	30.902	COCO train2017
val	1.024	5.479	5.390	COCO train2017, cắt ra 15%
test	310	1.592	1.568	COCO val2017
Tổng	7.137	38.460	37.860	

Hai cột instance khác nhau vì COCO đánh dấu một số vùng bằng cờ $iscrowd = 1$, đó là những đám đông vật thể được khoanh bằng một vùng duy nhất thay vì từng hộp riêng (nét đứt). Có 600 nhãn như vậy (487 train, 89 val, 24 test). CocoRecords bỏ chúng khi nạp dữ liệu huấn luyện, còn pycocotools xử lý riêng khi chấm điểm, nên cả hai con số đều đúng nhưng đếm hai thứ khác nhau. Cột phải là con số các mô hình thực sự nhìn thấy.

1.4 Thống kê bộ dữ liệu

1.4.1 Dữ liệu trông như thế nào

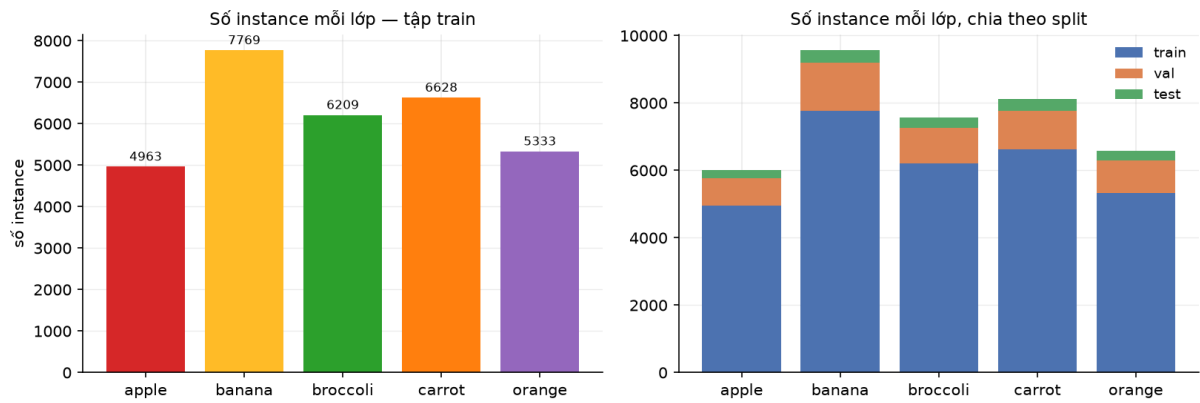


Hình 1: Ảnh mẫu kèm nhãn thật, tập train

Chín ảnh lấy ngẫu nhiên từ tập train, vẽ kèm nhãn thật (đọc trực tiếp từ `instances_train.json`).

Tiêu đề hình ghi quy ước hộp nét đứt dành cho vùng $iscrowd$ đã nói ở mục 1.3; chín ảnh được lấy ra lần này tình cờ không có vùng nào như vậy, nên mọi hộp đều nét liền (tỉ lệ 487 trên 31.389 nhãn train)

Hình cho thấy độ khó thật của bài toán: táo xếp chồng trong đĩa với ranh giới giữa các quả rất mờ, chuối trong nải dính liền nhau, cà rốt thái nhỏ rải trên mặt món ăn thành hàng chục mẫu li ti.



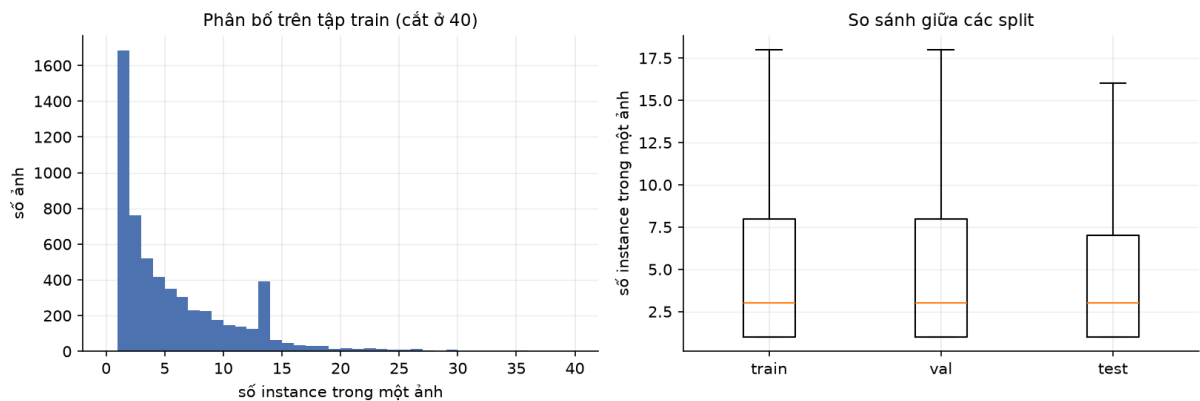
Hình 2: Số instance theo lớp

1.4.2 Số instance theo lớp

Lớp	train	val	test	Tổng	Tỉ lệ
apple	4.963	813	236	6.012	15,9%
banana	7.769	1.426	370	9.565	25,3%
broccoli	6.209	1.052	312	7.573	20,0%
carrot	6.628	1.130	365	8.123	21,5%
orange	5.333	969	285	6.587	17,4%

Lớp nhiều nhất (banana) gấp 1,6 lần lớp ít nhất (apple). Mức chênh lệch này không lớn, nhưng cần lưu ý: một lớp đạt điểm thấp có thể vì nó khó, cũng có thể vì nó ít dữ liệu, và bảng AP đơn thuần không tách được hai lý do đó.

1.4.3 Số vật thể trong một ảnh



Hình 3: Số instance trên mỗi ảnh

Chỉ số	train	val	test
Trung bình	5,33	5,26	5,06
Trung vị	3	3	3
Phân vị 90%	13	13	13
Lớn nhất	38	34	27

Phân bố lệch mạnh: ảnh điển hình chứa 3 vật thể, nhưng 10 % số ảnh chứa từ 13 vật thể trở lên, và ảnh dày nhất có 38.

1.4.4 Kích thước vật thể



Hình 4: Kích thước vật thể

COCO chia vật thể thành ba nhóm theo diện tích hộp, và chính cách chia này được `pycocotools` dùng lại khi tính AP theo kích thước:

Nhóm	Diện tích	Số instance (train)	Tỉ lệ
Nhỏ	Dưới 32×32 px	8.148	26,4%
Trung bình	32×32 đến 96×96 px	13.948	45,1%
Lớn	Trên 96×96 px	8.806	28,5%

Hơn một phần tư số vật thể thuộc nhóm nhỏ. Biểu đồ giữa cho thấy phần lớn hộp chiếm dưới 4 % diện tích ảnh. Biểu đồ phải cho thấy tỉ lệ khung hình tập trung quanh 1, rau củ quả phần lớn có hình dạng gần vuông, và cột dựng ở mốc 4 là phần đuôi bị cắt, gồm những bó cà rốt và nải chuối nằm ngang.

2 Ba kiến trúc được chọn

2.1 Lý do lựa chọn mô hình

Để đánh giá toàn diện hiệu năng phát hiện đối tượng, bài làm lựa chọn ba mô hình đại diện cho ba hướng tiếp cận kiến trúc tiêu biểu: CNN truyền thống, họ mô hình YOLO, và Vision Transformer. **Đồng thời, toàn bộ mô hình chọn với các phiên bản rút gọn (lightweight) nhằm đảm bảo khả năng triển khai thực tế trên các thiết bị IoT/edge có tài nguyên tính toán hạn chế.**

Mô hình	Thư viện	Kích thước vào	Số tham số
SSDLite320-MobileNetV3-Large	torchvision	320×320	2.261.960
YOLO11s	ultralytics 8.4.117	640×640	9.414.735
D-FINE-N	transformers 5.15.0	640×640	3.722.121

Bảng 1: Tổng quát các mô hình được lựa chọn.

2.2 Cơ chế của từng hướng

SSDLite320-MobileNetV3-Large. Backbone MobileNetV3 trích xuất đặc trưng đa quy mô trên sáu tầng phân giải khác nhau. Tại mỗi vị trí không gian trên từng tầng, mô hình khởi tạo các hộp neo (anchor boxes) với tỷ lệ cố định, đồng thời dự đoán đồng thời xác suất phân loại lớp và độ lệch tọa độ (offsets) so với nhãn thực tế. **Nhằm tối ưu hóa tài nguyên cho thiết bị nhỏ/IoT**, toàn bộ các lớp tích chập chuẩn ở nhánh dự đoán (head) được thay bằng tích

chập tách chiều sâu của biến thể SSDLite, giúp cắt giảm đáng kể chi phí tính toán (chỉ 2,26M tham số). Quá trình hậu xử lý sử dụng Non-Maximum Suppression (NMS) để loại bỏ các dự đoán dư thừa.

YOLO11s. Khác với tiếp cận dựa trên anchor, YOLO11s dự đoán trực tiếp khoảng cách từ từng điểm trên lưới đặc trưng tới bốn cạnh của bounding box (anchor-free). Trong quá trình huấn luyện, mô hình áp dụng cơ chế phân bổ mẫu *Task-Aligned Assigner*, kết hợp đồng thời độ tương quan hình học và điểm tin cậy phân loại để gán nhãn mục tiêu tối ưu. Mô hình tận dụng kỹ thuật tăng cường dữ liệu mạnh như Mosaic để tăng tính tổng quát hóa, và vẫn duy trì tầng NMS ở bước hậu xử lý để chọn lọc bounding box tối ưu.

D-FINE-N. Dựa trên kiến trúc Transformer phát hiện đối tượng đầu-cuối, mô hình sử dụng 300 truy vấn đối tượng có thể học được. Thông qua sáu tầng Decoder với cơ chế Self-Attention và Cross-Attention, mô hình trực tiếp ánh xạ ra đúng 300 dự đoán. Bằng cách ứng dụng thuật toán so khớp một-một khi huấn luyện, D-FINE-N hoàn toàn loại bỏ sự phụ thuộc vào NMS. Đặc biệt, cơ chế *FINE* nâng cao độ chính xác định vị bằng cách mô hình hóa việc tinh chỉnh tọa độ dưới dạng phân phối xác suất và làm mịn lặp qua từng tầng Decoder.

2.3 Checkpoint mô hình với trọng số COCO

Cả năm lớp của bài toán đều nằm trong 80 lớp COCO. Vì vậy thay vì khởi tạo ngẫu nhiên phần đầu phân loại, `src/models/head_transfer.py` sao chép đúng năm hàng trọng số tương ứng từ checkpoint COCO gốc sang mô hình 5 lớp.

Việc này có hai hệ quả:

1. Trọng số mô hình gốc có được từ việc pre-train trên bộ COCO 2017.
2. Thay vì huấn luyện từ đầu với 5 lớp, bài làm thực hiện kiểm tra xem liệu rằng fine-tune các mô hình với sub class nhỏ hơn (mặc dù đã được học tổng quát) xem hiệu năng có tốt hơn không.

2.4 Cấu hình huấn luyện

Tham số	SSDLite	YOLO11s	D-FINE-N
Epoch cấu hình	15	15	15
Epoch thực tế	15	15	15
Batch size	32	16	16
Số bước / epoch	181	363	362
Bộ tối ưu hóa	SGD (m=0,9, nesterov)	AdamW (auto)	AdamW
Learning rate	10^{-2}	$1,3 \times 10^{-4}$	$2,5 \times 10^{-4}$
Weight decay	4×10^{-5}	5×10^{-4}	1×10^{-4}
Warmup	3 epoch	3 epoch	2 epoch
LR Scheduler	Cosine	Cosine	Cosine
Early stopping	—	patience=20	—

Bảng 2: Cấu hình tham số huấn luyện của ba mô hình.

Cả ba dùng `workers=2`. Trên Windows, tiến trình nạp dữ liệu được sinh bằng `spawn` chứ không phải `fork`, nên mỗi worker phải nạp lại toàn bộ thư viện từ đầu; giá trị 2 là mức đo được ổn định nhất trên máy cá nhân (Strix i7 13th). Trên Linux con số này có thể nâng lên 8 được.

2.5 Thời gian huấn luyện

	SSDLite	YOLO11s	D-FINE-N
Tổng thời gian	29,7 phút	25,2 phút	42,2 phút
Thời gian trung vị mỗi epoch	96,5 s	83,5 s	140,2 s
VRAM đỉnh	1,81 GB	4,24 GB	3,09 GB

3 Kết quả trên tập test

3.1 Bảng tổng hợp

Mô hình	Tham số	Dung lượng	mAP _{50:95}	mAP ₅₀	mAP ₇₅	AR ₁₀₀	ms/ảnh	FPS
SSDLite320-MobileNetV3	2,26 M	8,9 MB	0,1238	0,2321	0,1179	0,2752	25,6	39,1
YOLO11s	9,41 M	18,3 MB	0,2642	0,4021	0,2896	0,5599	13,6	73,5
D-FINE-N	3,72 M	14,5 MB	0,0928	0,1790	0,0842	0,5044	42,5	23,5

Bảng 3: So sánh hiệu năng và tốc độ suy luận giữa ba mô hình thực nghiệm.

3.2 So với checkpoint COCO gốc

Mô hình	Checkpoint gốc	Sau huấn luyện	Chênh lệch
SSDLite320-MobileNetV3	0,1057	0,1238	+0,0181
YOLO11s	0,2814	0,2942	+0,0128
D-FINE-N	0,4715	0,3012	+0,0297

3.3 AR theo kích thước vật thể

Mô hình	AR nhỏ	AR trung bình	AR lớn
SSDLite320-MobileNetV3	0,0149	0,2451	0,5738
YOLO11s	0,3322	0,6156	0,6817
D-FINE-N	0,2101	0,5378	0,7243

AR nhỏ của SSDLite320 là 0,0149, nghĩa là nó không **tìm thấy** vật thể nhỏ, chứ không phải tìm thấy rồi chấm điểm sai. Còn D-FINE-N thì ngược lại: AR mỗi loại cao nhất trong ba mô hình, điều này chứng tỏ khả năng ”nhìn” của họ nhà Transformer tốt hơn hẳn so với các kiến trúc còn lại.

3.4 AP theo từng lớp

Mô hình	apple	banana	broccoli	carrot	orange
SSDLite320	0,0774	0,1345	0,1361	0,0756	0,1953
YOLO11s	0,2143	0,2556	0,2445	0,2590	0,3476
D-FINE-N	0,0843	0,1239	0,0743	0,0965	0,0850

Bảng 4: Độ chính xác trung bình (AP) chi tiết trên từng lớp đối tượng.

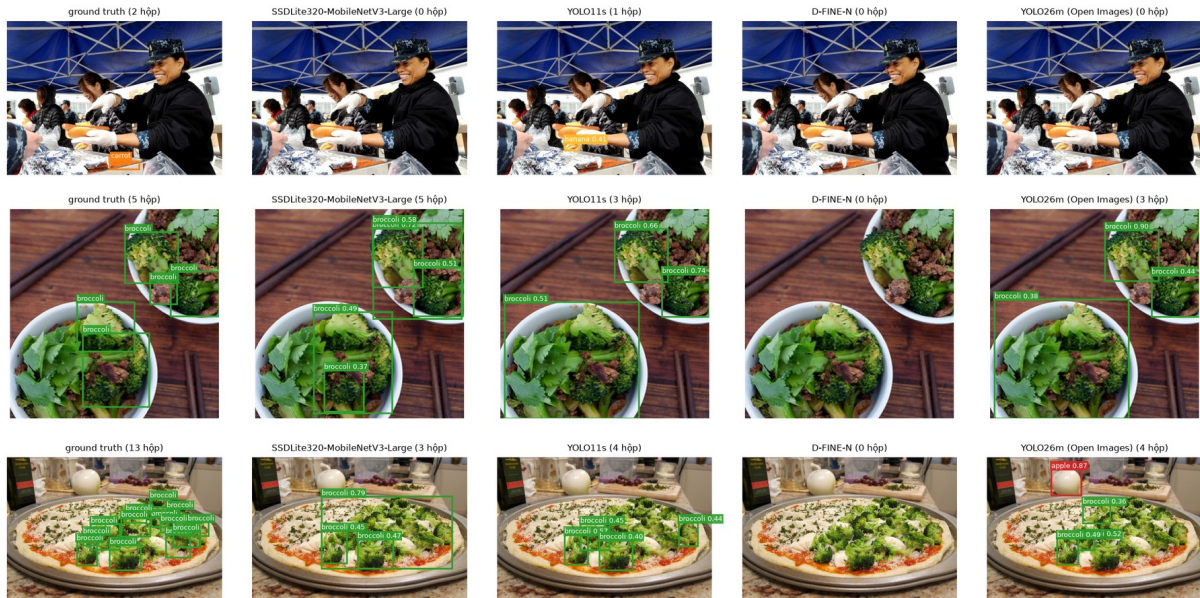
orange là lớp cao nhất ở cả SSDLite320 lẫn YOLO11s. Cam trong bộ dữ liệu này thường xuất hiện dưới dạng vật thể tròn, đơn lẻ, tương phản cao với nền. Ngược lại *apple* và *carrot* — hai lớp thấp nhất của SSDLite320 — thường nằm thành đồng trong sọt với ranh giới giữa các quả rất mờ, hoặc bị che khuất một phần và nằm bó chồng lên nhau. Đó là tình huống mà một tập anchor cố định khó tách từng thể hiện riêng, vì hai vật thể cạnh nhau rơi vào cùng một anchor.

YOLO11s trải đều hơn ($1,6\times$ so với $2,6\times$). Cơ chế gán nhãn theo độ khớp nhiệm vụ và tăng cường mosaic khiến nó gặp nhiều cấu hình chồng lẫn khác nhau hơn trong lúc huấn luyện.

D-FINE-N thấp đều ở cả năm lớp, không lớp nào nổi bật. Việc suy giảm xảy ra đồng đều chứ không tập trung vào một lớp cụ thể cho thấy nó không gắn với đặc điểm hình ảnh của một loại rau củ nào.

3.5 Quan sát trên tập test

Phát hiện trên tập test — ngưỡng score 0.35



Hình 5: Phát hiện trên tập test

Ba ảnh được chọn theo phân vị mật độ hộp (thưa – trung bình – dày) thay vì lấy các ảnh dày nhất. Lý do: những ảnh dày nhất trong tập test là quầy chợ 25 hộp và ảnh ghép lưới mà không mô hình nào xử lý được, chúng nói về đặc điểm của bộ dữ liệu nhiều hơn là nói về mô hình. Ngưỡng hiển thị 0,35, giống ngưỡng mặc định của ứng dụng web.

Hàng trên (2 hộp thật, một củ cà rốt nhỏ ở góc quầy hàng): SSDLite320 không đưa ra hộp nào, YOLO11s đưa ra 1 hộp nhưng gán nhầm nhãn *banana* cho ổ bánh mì. Đây là ảnh khó, vật thể nhỏ, bị che, nằm lệch góc.

Hàng giữa (5 hộp thật, các cụm bông cải trong bát): SSDLite320 đưa ra đúng 5 hộp; YOLO11s đưa ra 3 hộp nhưng gom hai cụm nhỏ liền kề thành một hộp lớn. Đây là một trong số ít trường hợp mà mô hình nhỏ hơn tách cụm tốt hơn, và nó khớp với việc SSDLite320 đặt anchor dày trên các tầng đặc trưng có độ phân giải cao.

Hàng dưới (13 hộp thật, bông cải rải trên mặt bánh pizza): SSDLite320 đưa ra 3 hộp, YOLO11s đưa ra 4. Cả hai đều gom nhiều bông cải nhỏ liền kề thành một hộp lớn thay vì tách riêng. Đây là biểu hiện thị giác của đúng hiện tượng mà cột AP-nhỏ ở mục 4.3 đã đo được bằng số.

Các ô của D-FINE-N trông ở cả ba hàng. Đây không phải do mô hình không sinh ra dự đoán — nó vẫn xuất đủ 100 hộp mỗi ảnh, tổng cộng 31.000 hộp trên toàn tập test, mà do toàn bộ điểm tin cậy đều nằm dưới ngưỡng 0,35. Mục 5.6 có con số cụ thể.

4 Đánh giá từng kiến trúc

4.1 SSDLite320-MobileNetV3-Large

Điểm yếu về độ chính xác nằm gọn ở một chỗ: **AP vật thể nhỏ 0,0013, và AR vật thể nhỏ 0,0149**. Mô hình gần như không tìm thấy vật thể nhỏ.

Tốc độ. 25,6 ms mỗi ảnh, chậm hơn YOLO11s gần gấp đôi, dù số tham số ít hơn bốn lần. Nguyên nhân là tích chập tách theo chiều sâu: nó tiết kiệm tham số bằng cách tách một phép tích chập thành hai phép nhỏ, nhưng hệ quả là chia phép tính thành rất nhiều nhân GPU nhỏ, mỗi nhân không lấp đầy được các đơn vị tính toán. Số lần phóng nhân tăng lên trong khi mỗi lần làm ít việc hơn. Trên Windows với trình điều khiển WDDM thì chi phí mỗi lần phóng nhân lại càng đáng kể.

Độ phức tạp huấn luyện. Thấp nhất trong ba: 29,7 phút cho 15 epoch, VRAM đỉnh 1,81 GB. Đây là mô hình duy nhất huấn luyện được thoải mái trên GPU 4 GB. Nó cũng là mô hình duy nhất mà đường cong val đi lên đều đặn từ đầu tới cuối mà không cần điều chỉnh gì.

Khả năng ứng dụng. Checkpoint 8,9 MB và VRAM 1,81 GB làm nó phù hợp với thiết bị IoT nhỏ, đúng mục đích thiết kế của MobileNet.

4.2 YOLO11s

Độ chính xác. mAP 0,2642, cao nhất trong ba mô hình, và nó dẫn đầu ở mọi chỉ số con: mAP@.5, mAP@.75, AR@100, AP nhỏ, AP trung bình, AP lớn. Nó cũng là mô hình trái đều nhất giữa năm lớp (chênh 1,6× giữa lớp cao nhất và thấp nhất).

Tốc độ. 13,6 ms, nhanh nhất trong ba, dù có nhiều tham số nhất. Các khối tích chập của nó lớn và được hợp nhất (BatchNorm gộp vào convolution sau khi huấn luyện xong), nên GPU chạy hết công suất trong ít lần phóng nhân. Đây là ví dụ rõ cho việc số tham số không quyết định độ trễ trên GPU.

Khả năng ứng dụng. Điểm cao nhất, tốc độ nhanh nhất, checkpoint 18,3 MB, hệ sinh thái xuất khẩu sẵn sang ONNX/TensorRT/CoreML. Đây là lựa chọn ít rào cản nhất trong ba mô hình cho một ứng dụng thực tế.

4.3 D-FINE-N

Độ chính xác. Cần đọc theo hai mốc riêng biệt.

Ở trạng thái gốc: 0,2715 — gần bằng YOLO11s (0,2814) trong khi chỉ dùng 3,72 triệu tham số so với 9,41 triệu, tức 40 %. Đây là bằng chứng cho thấy kiến trúc này mạnh, và cũng là bằng chứng cho thấy phần khởi tạo trọng số trong dự án hoạt động đúng.

Sau khi huấn luyện lại: 0,0928, tức còn một phần ba. Toàn bộ phần suy giảm đến từ quá trình huấn luyện, không phải từ kiến trúc.

Tốc độ. 42,5 ms, chậm nhất trong ba. Đổi lại, nó không cần NMS, nên độ trễ không phụ thuộc vào số vật thể trong ảnh: một ảnh có 1 vật thể và một ảnh có 40 vật thể tốn đúng như nhau. Hai mô hình kia thì chậm dần khi ảnh đầy hộp.

Độ phức tạp huấn luyện. Cao nhất trong ba. Nó cần nhiều tham số cấu hình tinh tế hơn: learning rate riêng cho backbone, cắt gradient rất chặt (0,1 so với 10,0 của SSDLite).

4.4 Huấn luyện lại trên tập con của dữ liệu tiền huấn luyện làm mô hình tệ đi?

Mặc dù kết quả ở Mục 3.2 cho thấy điểm số của ba mô hình vẫn nhích nhẹ sau khi huấn luyện lại, mức cải thiện này không phản ánh sự vượt trội về năng lực học mà thực chất là sự đánh đổi về biểu diễn đặc trưng.

Các trọng số tiền huấn luyện (như YOLO11s) vốn đã được tối ưu trên toàn bộ tập COCO (118.000 ảnh, 80 lớp). Việc huấn luyện lại trên tập con 5.803 ảnh không cung cấp thêm bất kỳ phân phối dữ liệu mới nào. Dù mô hình có thể tăng nhẹ điểm số nhờ sự thích nghi tạm thời với phân phối của tập con, nó đồng thời chịu tổn thất lớn do:

- Thu hẹp miền dữ liệu làm suy yếu không gian biểu diễn dùng chung giữa các lớp đã học trước đó.
- Gradient liên tục cập nhật trên tập mẫu nhỏ dễ kéo trọng số lệch khỏi điểm tối ưu toàn cục ban đầu.

Giả thuyết về tác động bất lợi khi huấn luyện trên tập con sẽ được kiểm chứng độc lập ở Chương 6, thông qua thực nghiệm với dữ liệu mới mà checkpoint chưa từng tiếp xúc.

4.5 Khoảng cách giữa khả năng tìm và khả năng chấm điểm

Đặt cạnh nhau AR@100 (khả năng tìm ra vật thể) và mAP (có tính đến độ tin cậy và độ khớp của hộp):

Mô hình	YOLO26m (checkpoint COCO, 20,4M tham số)
Dữ liệu	Open Images v7 (5 lớp), tải qua CSV và S3 bucket
Kích thước	Train: 2.755 ảnh (10.261 mẫu) / Val: 567 ảnh (2.182 mẫu)
Phần cứng	Kaggle, 2× Tesla T4
Huấn luyện	60 epoch, 123,3 phút, VRAM tối đa 9,04 GB
val mAP	0,3335 (mAP@0.5:0.95)

SSDLite320 và YOLO11s có tỉ lệ khoảng 2×, là mức thông thường. D-FINE-N thì khác hẳn: nó **tìm ra vật thể ở mức tốt** — AR 0,5044, gần bằng YOLO11s và cao gần gấp đôi SSDLite320 — nhưng mAP chỉ bằng một phần năm AR.

Chênh lệch này chỉ ra vị trí của vấn đề. Nếu mô hình không định vị được vật thể thì AR phải thấp; AR 0,50 cho thấy các truy vấn của nó vẫn phủ đúng vùng có vật thể. Thứ suy giảm là điểm tin cậy và độ khớp của hộp: mAP@.75 (0,0842) chỉ bằng 47 % mAP@.5 (0,1790), nghĩa là hộp phủ đúng vật thể nhưng lệch nhiều khi yêu cầu độ khớp chặt hơn.

Một phép đo cụ thể hoá điều này. Chạy lại D-FINE-N trên toàn bộ 310 ảnh test và lấy điểm tin cậy cao nhất mà nó từng đưa ra:

Checkpoint	Điểm tin cậy cao nhất trên toàn tập test	Số hộp vượt ngưỡng 0,35	Số ảnh có ít nhất một hộp
best	0,3028	0	0 / 310
last	0,3342	0	0 / 310

5 Thử nghiệm mở rộng: YOLO26m trên Open Images

Chương này tách riêng khỏi phần so sánh ba kiến trúc ở chương 4–5, vì mô hình ở đây được huấn luyện trên **một bộ dữ liệu khác** và được thực hiện sau (lựa chọn thêm), nên không so sánh trực tiếp với ba mô hình kia về mặt huấn luyện. Chỉ phần chấm điểm trên tập test COCO là so sánh được.

5.1 Cấu hình

Mô hình	YOLO26m (checkpoint COCO, 20,4M tham số)
Dữ liệu	Open Images v7 (5 lớp), tải qua CSV và S3 bucket
Kích thước	Train: 2.755 ảnh (10.261 mẫu) / Val: 567 ảnh (2.182 mẫu)
Phần cứng	Kaggle, 2× Tesla T4
Huấn luyện	60 epoch, 123,3 phút, VRAM tối đa 9,04 GB
val mAP	0,3335 (mAP@0.5:0.95)

5.2 Kết quả

Mô hình được chấm trên **hai** tập:

	COCO test (310 ảnh)	Open Images val
YOLO26m gốc	0,3063	0,4659
YOLO26m sau huấn luyện	0,1353	0,3671
Chênh lệch	−0,1710	−0,0988

5.3 So sánh với ba mô hình chính

Chấm trên cùng tập test COCO, cùng bộ đo, cùng lượt đo tốc độ.

Mô hình	Tham số	mAP (gốc)	AR@100	Độ trễ (ms/ảnh)
YOLO26m	20,35 M	0,3063	0,3411	16,9
YOLO11s	9,41 M	0,2814	0,5599	13,6
SSDLite320	2,26 M	0,1057	0,2752	25,6
D-FINE-N	3,72 M	0,2715	0,5044	42,5

Bảng 5: So sánh hiệu năng gốc và độ trễ suy luận giữa các mô hình.

6 Ứng dụng web

6.1 Mô hình được chọn

Ứng dụng dùng **yolo26m.pt ở trạng thái gốc**. Cao hơn mọi phương án khác đã đo trong dự án, kể cả mọi phương án sau huấn luyện. Với 16,9 ms mỗi ảnh, nó đủ nhanh cho chức năng của ứng dụng.

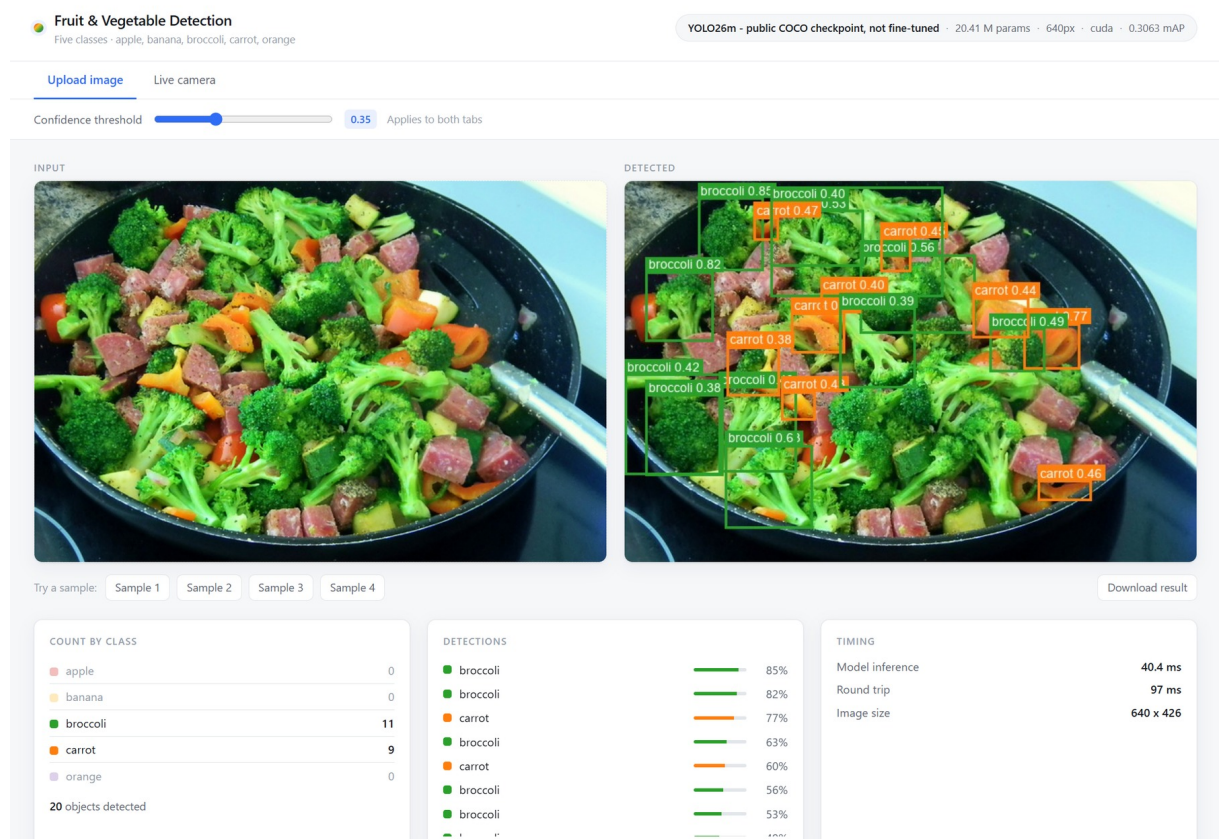
6.2 Kiến trúc hệ thống

Backend là FastAPI, frontend là JavaScript thuần.

File	Việc
src/webapp/detector.py	Nạp mô hình, suy luận, đếm số lượng, bám vết
src/webapp/session.py	Phiên camera: đếm vật thể riêng biệt, ghi JSON và CSV
src/webapp/drawing.py	Vẽ hộp bằng PIL, mã hoá data URL
src/webapp/app.py	FastAPI: lifespan và các route
src/webapp/cli.py	argparse và uvicorn
webapp/frontend/	index.html + static/{app.js, style.css}

Route	Việc
GET /	Trang HTML
GET /api/meta	5 lớp, bảng màu, ngưỡng mặc định, thông tin mô hình, danh sách ảnh mẫu
POST /api/detect	Ảnh + ngưỡng → JSON kèm ảnh đã vẽ hộp (data URL), danh sách vật thể, số lượng theo lớp
WS /ws/detect	Khung JPEG vào, chỉ tọa độ ra — trình duyệt tự vẽ overlay lên canvas
GET /api/sessions	Danh sách phiên camera đã lưu, mới nhất trước
GET /sessions/*	File JSON từng phiên và sessions.csv

6.3 Chức năng 1: tải ảnh lên và trả kết quả



Hình 6: Ứng dụng web, tab Upload

Người dùng kéo thả ảnh hoặc bấm một ảnh mẫu. Kết quả gồm: ảnh gốc và ảnh đã vẽ hộp đặt cạnh nhau, bảng đếm theo lớp, danh sách từng vật thể kèm điểm tin cậy, và bảng thời gian. Thanh trượt ngưỡng tin cậy áp dụng cho cả hai tab.

6.4 Chức năng 2: camera thời gian thực và giả lập đếm bằng chuyên

Trình duyệt gửi từng khung JPEG qua WebSocket, nhận về tọa độ hộp, và tự vẽ overlay lên canvas đặt chồng lên thẻ `<video>`.

Cơ chế: client chỉ gửi khung kế tiếp sau khi nhận được kết quả của khung trước. Điều này giữ độ trễ ổn định thay vì để hàng đợi phình ra khi GPU chậm hơn camera. Hệ quả kỹ thuật: server **phải trả lời mọi khung, kể cả khung hỏng** — bỏ qua một khung mà không trả lời sẽ làm camera đứng im vĩnh viễn chứ không phải mất một khung.

Đếm theo id riêng biệt của tracker, không cộng số đếm từng khung. Đây là quyết định quan trọng nhất của phần này. Nếu cộng số đếm từng khung thì ở 30 fps, một quả táo nằm yên 10 giây sẽ được đếm thành 300 — con số đó đo tốc độ khung hình chứ không đo trái cây. Tracker cấp một id khi vật thể xuất hiện và giữ id đó suốt thời gian nó còn trong khung; rồi đi rồi quay lại là id mới, tức vật thể mới. Tổng phiên là phép hợp tập hợp các id, nên cùng một quả xuất hiện trong 300 khung vẫn chỉ vào tập một lần.

6.5 Lưu và xem lại các phiên

Một phiên bắt đầu khi bấm Start camera và kết thúc khi bấm Stop. **Socket chính là phiên** , không cần cơ chế đồng bộ nào thêm, và phiên kết thúc vì đóng tab hay mất mạng vẫn được ghi, vì đường ngắt kết nối chạy chung hàm

Fruit & Vegetable Detection
 Five classes - apple, banana, broccoli, carrot, orange

YOLO26m - public COCO checkpoint, not fine-tuned · 20.41 M params · 640px · cuda · 0.3063 mAP

Upload image

Live camera

Confidence threshold

0.35

 Applies to both tabs

CAMERA

Stop camera

live

THROUGHPUT
 Frames per second: 19.7
 Model inference: 21.0 ms
 Round trip: 32 ms
 Frames processed: 30

COUNT BY CLASS

apple	0
banana	0
broccoli	8
carrot	6
orange	0

 14 objects in frame

SESSION TOTAL

apple	0
banana	0
broccoli	8
carrot	6
orange	0

 14 distinct objects this session
 Started 3:51:12 AM · 2 s · 30 frames

SAVED SESSIONS

Download sessions.csv

Hình 7: Ứng dụng web, tab Live

SAVED SESSIONS

Download sessions.csv

STARTED	DURATION	FRAMES	COUNTED	TOTAL	FILE
8/23/2026, 3:51:12 AM	3 s	50	8 broccoli 6 carrot	14	JSON
8/23/2026, 3:50:10 AM	3 s	49	8 broccoli 6 carrot	14	JSON
8/23/2026, 3:49:12 AM	3 s	46	8 broccoli 6 carrot	14	JSON
8/22/2026, 8:27:28 AM	3 s	120	3 apple	3	JSON
8/22/2026, 8:27:25 AM	3 s	120	7 apple	7	JSON
8/22/2026, 8:27:23 AM	2 s	90	3 apple	3	JSON
8/22/2026, 8:26:49 AM	3 s	150	6 apple	8	JSON
8/22/2026, 8:26:46 AM	2 s	120	6 apple	8	JSON
8/22/2026, 8:26:43 AM	3 s	120	4 apple	4	JSON
8/22/2026, 8:26:40 AM	2 s	90	4 apple	4	JSON
8/22/2026, 8:25:50 AM	3 s	120	4 apple	4	JSON
8/22/2026, 8:25:47 AM	3 s	90	1 apple	1	JSON
8/22/2026, 8:25:44 AM	3 s	90	3 apple	3	JSON
8/22/2026, 8:25:42 AM	3 s	90	nothing detected	0	JSON
8/22/2026, 8:25:39 AM	3 s	90	1 apple	1	JSON
8/22/2026, 8:21:49 AM	4 s	120	2 orange	2	JSON
8/22/2026, 8:21:45 AM	3 s	120	2 orange	2	JSON
8/22/2026, 8:21:41 AM	4 s	150	1 orange	1	JSON
8/22/2026, 8:21:36 AM	4 s	150	2 orange	2	JSON
8/22/2026, 8:21:32 AM	4 s	150	1 orange	1	JSON

Each session is written to `webapp/sessions/` as a JSON file when it ends, and appended as one row to `sessions.csv`. Counting is by distinct tracker id, so an object that stays in view is counted once, and one that leaves and comes back counts again.

Hình 8: Bảng phiên đã lưu

close().

Mỗi phiên ghi ra `webapp/sessions/session_<id>.json`, và nối thêm một dòng vào `sessions.csv` để so sánh giữa các phiên:

```
{"session_id": "20260822_081013",
"started_at": "2026-08-22T08:10:13+07:00",
"ended_at": "2026-08-22T08:10:18+07:00",
"duration_seconds": 4.9, "frames": 146,
"counts": {"apple": 4, "banana": 10, "broccoli": 0, "carrot": 0, "orange": 8},
"total": 22}
```

6.6 Số đo hiệu năng

Đo trên RTX 4060 Laptop.

Chế độ / Kích bản	Kích thước ảnh (imgsz)	Độ trễ trung bình	FPS ước tính
Vòng lặp liên tục	320 / 480 / 640	$\approx 21,0$ ms	$\approx 47,6$
Tab Live	640	27,7 ms	36,1
Tab Upload	640	40,4 ms	24,8

Bảng thời gian hiển thị trong hai ảnh chụp màn hình ở trên cho những con số khác: 27,7 ms ở tab Live, 40,4 ms ở tab Upload. Chênh lệch đến từ đúng hiện tượng: tab Live giữ GPU bận liên tục nên nó chạy ở xung cao, còn tab Upload gửi từng request rồi rạc nên giữa các request GPU tụt xuống xung thấp. Con số đại diện là hàng đầu của bảng, đo trong vòng lặp liên tục.

GPU máy cá nhân đủ để chạy ứng dụng này, nên không cần đến GPU server riêng. Ba mức `imgsz` 320 / 480 / 640 cho cùng một thời gian, nghĩa là phần lớn trong ~ 21 ms đó là tiền xử lý và hậu xử lý phía CPU của ultralytics chứ không phải phép tính trên GPU.

Phương án chạy ONNX trong trình duyệt đã được cân nhắc và bỏ: nó đòi viết lại letterbox và bộ giải mã đầu ra YOLO26 bằng JavaScript.

Lời cảm ơn

Em xin gửi lời cảm ơn chân thành đến thầy Ngô Minh Nhật và thầy Lê Long Quốc, hai giảng viên phụ trách môn học, đã tận tình hướng dẫn, truyền đạt kiến thức, giải đáp thắc mắc trong suốt học kỳ vừa qua.

Và cũng xin cảm ơn Claude Code đã hỗ trợ xây dựng web trong project này, thiết kế các scripts kiểm thử và chỉnh sửa lỗi. Cảm ơn Gemini đã hỗ trợ xây dựng báo cáo.

Tài liệu

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [2] G. Jocher and J. Qiu, “Ultralytics YOLO11: Real-time Object Detection and Segmentation,” *Ultralytics Software Library*, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>

- [3] G. Jocher and J. Qiu, “Ultralytics YOLO26: Next-Generation Real-Time Vision Framework,” *Ultralytics Software Library*, 2026. [Online]. Available: <https://docs.ultralytics.com>
- [4] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.
- [5] Y. Peng, S. Shen, M. Cheng, P. Gao, X. Lv, S. Wei, and Z. Zou, “D-FINE: Redefine Bounding Box Regression for DETR-based Object Detection,” *arXiv preprint arXiv:2410.13842*, 2024.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” in *European Conference on Computer Vision (ECCV)*, pp. 740–755, Springer, 2014.
- [7] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Mallocci, A. Kolesnikov, T. Duerig, and V. Ferrari, “The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale,” *International Journal of Computer Vision (IJCV)*, vol. 128, no. 7, pp. 1956–1981, 2020.
- [8] B. E. Moore and J. J. Corso, “FiftyOne: A Tool for Building High-Quality Datasets and Computer Vision Models,” *Voxel51*, 2020. [Online]. Available: <https://github.com/voxel51/fiftyone>
- [9] ONNX Runtime Developers, “Open Neural Network Exchange (ONNX): Open Format to Represent Deep Learning Models,” *Linux Foundation / ONNX Runtime Project*, 2021. [Online]. Available: <https://onnx.ai>
- [10] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in *European Conference on Computer Vision (ECCV)*, pp. 1–21, Springer, 2022.
- [11] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.
- [12] S. Ramírez, “FastAPI: High Performance Web Framework for Building APIs with Python,” *GitHub repository*, 2018. [Online]. Available: <https://fastapi.tiangolo.com>